

Wireshark Network Traffic Analysis Report



Wireshark-Based Network Traffic Analysis and Security Monitoring

University: University of Colombo School of Computing

Tool: Wireshark

Operating System: Windows 11

Date: September 2026

2. Introduction

This project focuses on learning and analyzing network traffic using Wireshark. Wireshark is a network protocol analyzer that allows network packets to be captured and examined in detail. Through this project, different network protocols and communication processes were observed, including ARP, ICMP, DNS, TCP, HTTP/HTTPS, and TLS.

The main purpose of this project was to understand how network communication occurs at the packet level and to identify information that can be obtained from captured network traffic. The project also introduced basic security analysis by examining patterns that may indicate suspicious network activity.



3. Objectives

Main objective

To gain practical knowledge of network traffic analysis and basic network security monitoring using Wireshark.

Objectives

1. Learn how to capture network packets using Wireshark.
2. Understand the structure and information contained in network packets.
3. Analyze common network protocols.
4. Understand the TCP three-way handshake.
5. Analyze DNS queries and responses.
6. Observe ARP communication.
7. Analyze ICMP traffic generated using ping.
8. Understand the difference between HTTP and HTTPS traffic.
9. Identify IP addresses, ports, protocols, and TCP flags from captured packets.
10. Investigate patterns that may indicate suspicious network activity.

4. Tools and Environment

Item	Details
Operating System	Windows
Network connection	Wi-Fi
Main tool	Wireshark
Command Prompt	Windows CMD
Capture format	.pcapng
Test device	Personal Windows laptop
Internet connection	Personal network

5. What is Wireshark?

5.1 Overview

Wireshark is a network protocol analyzer used to capture and inspect network packets. It provides detailed information about network communication, including source and destination addresses, protocols, ports, packet lengths, TCP flags, and protocol-specific information.

5.2 Why Wireshark is useful

- Network troubleshooting
- Protocol analysis
- Network security investigations
- Identifying unusual traffic
- Understanding communication between systems
- Troubleshooting connectivity problems
- Learning networking concepts

6. Understanding a Network Packet

No.	Time	Source	Destination	Protocol	Length	Info
343	65.142415	192.168.0.21	174.129.249.228	TCP	66	40555 → 80 [ACK] Seq=1 Ack=1 Win=5888 Len=0 TSval=491519346 TSecr=551811827
344	65.142715	192.168.0.21	174.129.249.228	HTTP	253	GET /clients/netflix/flash/application.swf?flash_version=flash_lite_2.1&v=1.5&n...
345	65.230738	174.129.249.228	192.168.0.21	TCP	66	80 → 40555 [ACK] Seq=1 Ack=188 Win=6864 Len=0 TSval=551811850 TSecr=491519347
346	65.240742	174.129.249.228	192.168.0.21	HTTP	828	HTTP/1.1 302 Moved Temporarily
347	65.241592	192.168.0.21	174.129.249.228	TCP	66	40555 → 80 [ACK] Seq=188 Ack=763 Win=7424 Len=0 TSval=491519446 TSecr=551811852
348	65.242532	192.168.0.21	192.168.0.1	DNS	77	Standard query 0x2188 A cdn-0.nflximg.com
349	65.276870	192.168.0.1	192.168.0.21	DNS	489	Standard query response 0x2188 A cdn-0.nflximg.com CNAME images.netflix.com.edg...
350	65.277992	192.168.0.21	63.80.242.48	TCP	74	37063 → 80 [SYN] Seq=0 Win=5840 Len=0 MSS=1460 SACK_PERM=1 TSval=491519482 TSecr=...
351	65.297757	63.80.242.48	192.168.0.21	TCP	74	80 → 37063 [SYN, ACK] Seq=0 Ack=1 Win=5792 Len=0 MSS=1460 SACK_PERM=1 TSval=3295...
352	65.298396	192.168.0.21	63.80.242.48	TCP	66	37063 → 80 [ACK] Seq=1 Ack=1 Win=5888 Len=0 TSval=491519502 TSecr=3295534130
353	65.298687	192.168.0.21	63.80.242.48	HTTP	153	GET /us/nrd/clients/flash/814540.bun HTTP/1.1
354	65.318730	63.80.242.48	192.168.0.21	TCP	66	80 → 37063 [ACK] Seq=1 Ack=88 Win=5792 Len=0 TSval=3295534151 TSecr=491519503
355	65.321733	63.80.242.48	192.168.0.21	TCP	1514	[TCP segment of a reassembled PDU]

> Frame 349: 489 bytes on wire (3912 bits), 489 bytes captured (3912 bits)

> Ethernet II, Src: Globalsc_00:3b:0a (f0:ad:4e:00:3b:0a), Dst: Vizio_14:8a:e1 (00:19:9d:14:8a:e1)

> Internet Protocol Version 4, Src: 192.168.0.1, Dst: 192.168.0.21

> User Datagram Protocol, Src Port: 53 (53), Dst Port: 34836 (34836)

Domain Name System (response)

[Request In: 348]

[Time: 0.034338000 seconds]

Transaction ID: 0x2188

> Flags: 0x8180 Standard query response, No error

Questions: 1

Answer RRs: 4

Authority RRs: 9

Additional RRs: 9

Queries

> cdn-0.nflximg.com: type A, class IN

Answers

Authoritative nameservers

0020 00 15 00 35 84 f4 01 c7 83 3f 21 00 81 80 00 01 ...5....?M....
0030 00 04 00 09 00 09 05 63 64 6e 2d 30 07 6e 66 6cc dn-0.nfl
0040 78 69 6d 67 03 63 6f 6d 00 00 01 00 01 c0 0c 00 ximg.com
0050 05 00 01 00 00 05 29 00 22 06 09 6d 61 67 65 73). ".images
0060 07 6e 65 74 66 6c 69 78 03 63 6f 6d 09 65 64 67 .netflix .com.edg
0070 65 73 75 69 74 65 03 6e 65 74 00 c0 2f 00 05 00 esuite.n et.../....

Packet List

Explain:

The packet list displays captured packets and provides basic information such as packet number, time, source, destination, protocol, length, and additional information.

Packet Details

Explain:

The packet details section provides a hierarchical view of the protocols and fields contained within the selected packet.

Packet Bytes

Explain:

The packet bytes section displays the raw hexadecimal and ASCII representation of the captured packet.

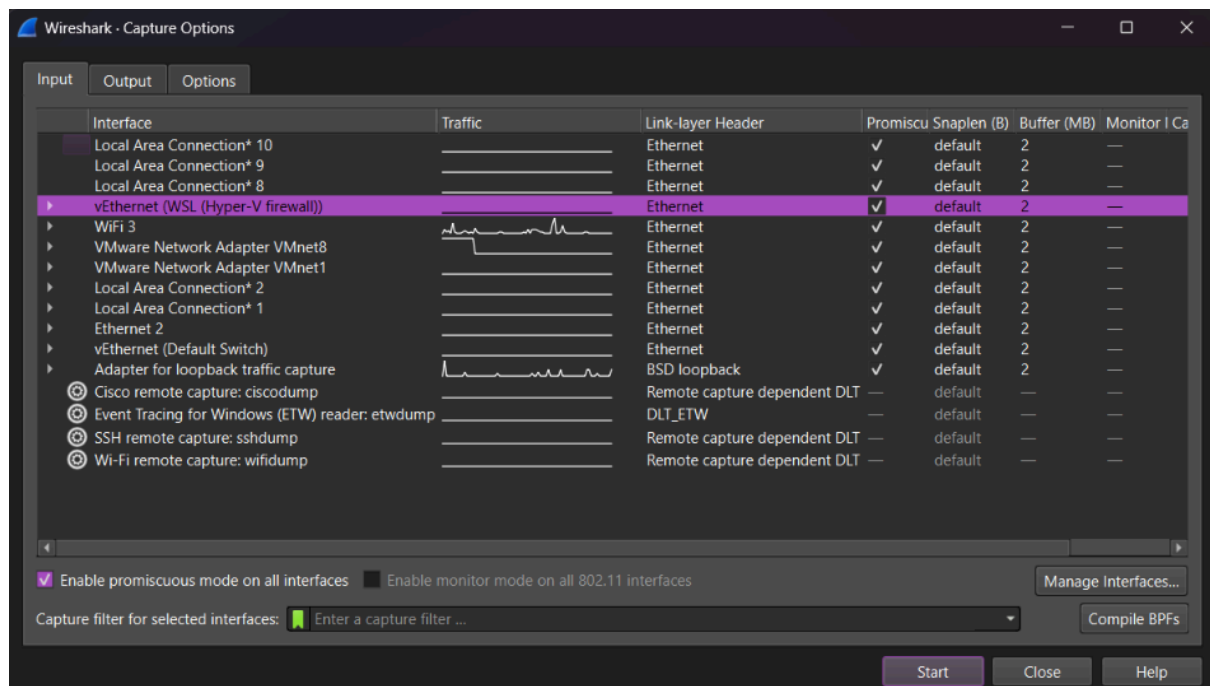


Figure 1: Wireshark packet analysis interfaces

7. Experiment 01 - ICMP Analysis

7.1 Objective

To understand ICMP communication by generating ping traffic and analyzing the resulting packets.

7.2 Procedure

Open Command Prompt:

`ping google.com`

Then use the Wireshark filter:

`icmpv6`

7.3 Observation

You should see ICMP packets.

Typically:

Echo Request

Echo Reply

Explain:

The ICMP Echo Request is sent from the local system to the destination, and the Echo Reply is returned by the destination. This demonstrates how the ping command can be used to test network connectivity.

7.4 Screenshot

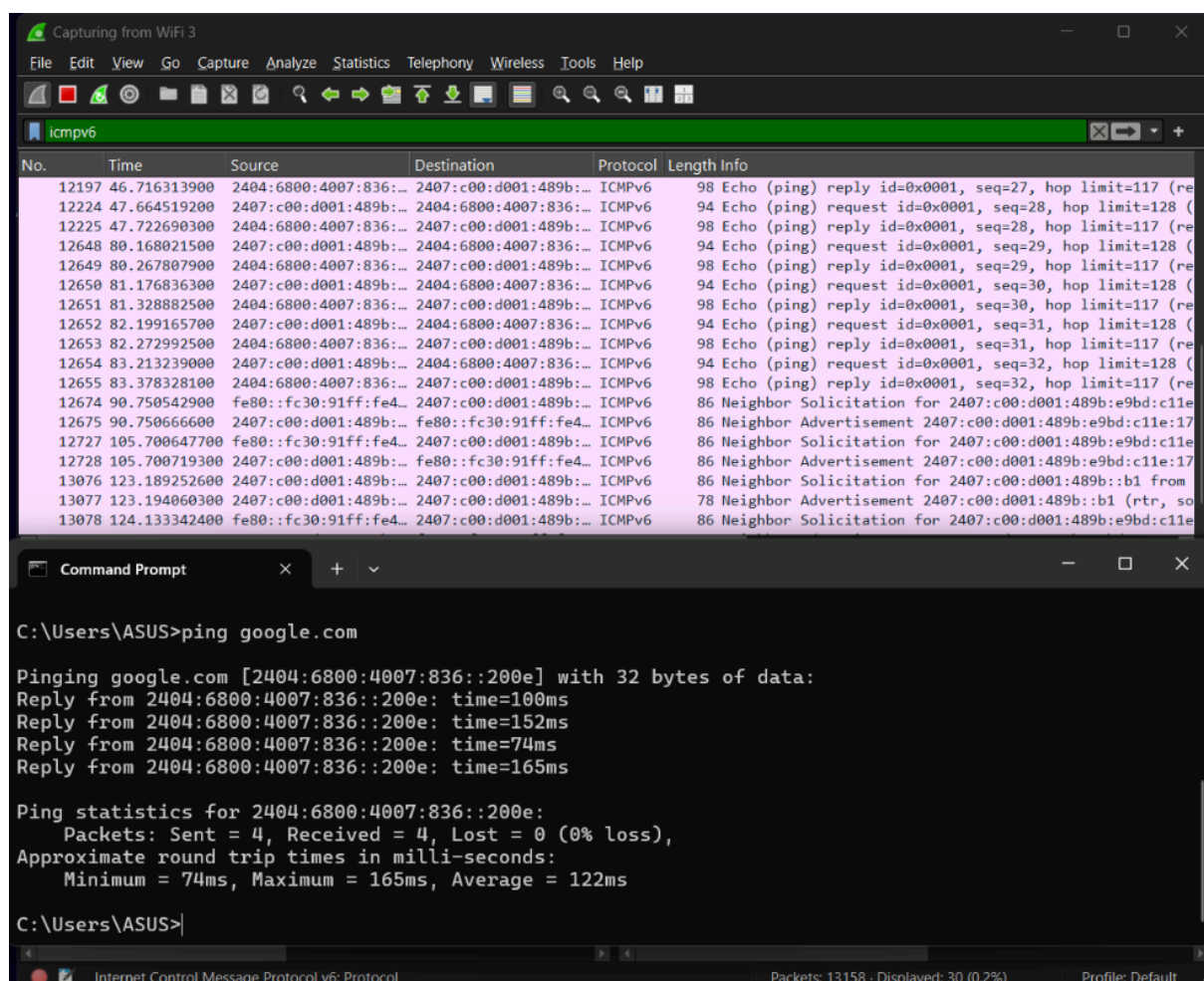


Figure 2: ICMP Echo Request and Echo Reply packets

7.5 Learning outcome

From this experiment, I learned how ICMP is used for network connectivity testing and how ICMP request and response packets can be identified using Wireshark.

It is the "Return Receipt" for Ping: When you use Ping, you want to know if a computer is awake. ICMP is used because it has a built-in "Echo" feature. Your computer sends an ICMP packet saying, *"Repeat this back to me,"* and the other computer immediately bounces it back.

It is the "Return to Sender" stamp for Errors: If a router along the way finds out a website is dead, it can't use regular internet traffic to tell you. It uses ICMP to send an error message back to your computer saying, *"Destination Unreachable."*

It acts like a GPS tracker for Traceroute:

When you want to see the path your data takes, ICMP intentionally "expires" packets at each router along the way. Each router then sends an ICMP message back, letting you map the exact route.

8. Experiment 02 - DNS Analysis

Objective

To understand how domain names are resolved into IP addresses.

Procedure

Run:

```
nslookup example.com
```

Wireshark filter:

dns

Observation

Look for:

DNS Query & DNS Response

Explain:

When a domain name is requested, the DNS client sends a query to a DNS server. The DNS server responds with information associated with the requested domain, such as its IP address.

Screenshot

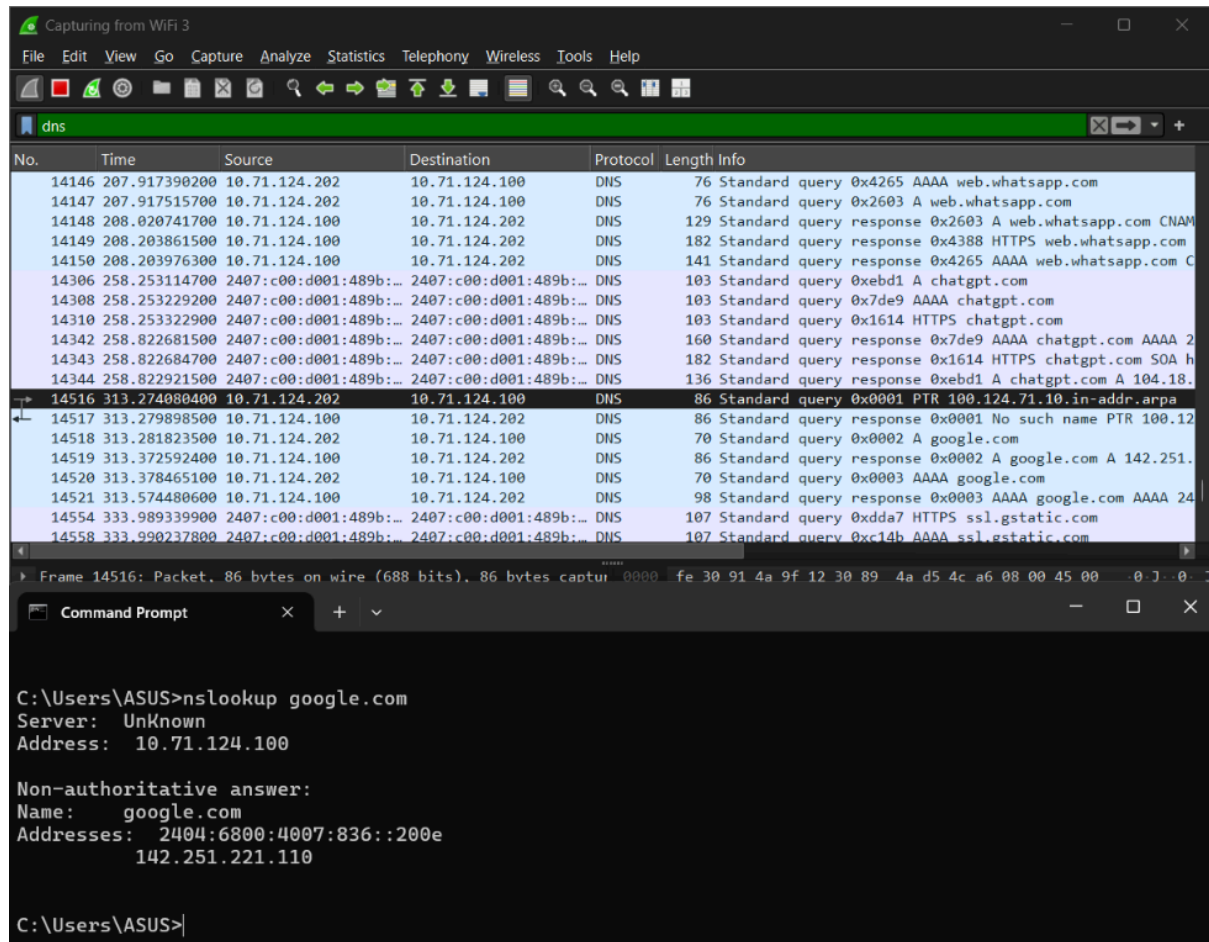


Figure 3: DNS query and response

What I learned

I learned that DNS acts as a mechanism for resolving human-readable domain names into IP addresses and that DNS queries and responses can be observed at packet level.

9. Experiment 03 - ARP Analysis

Objective

To understand how ARP is used to determine the MAC address associated with an IP address on a local network.

Wireshark filter: **arp**

Look for:

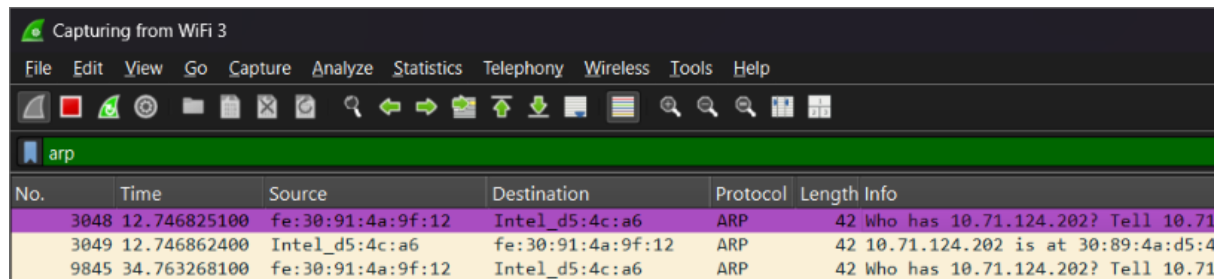
Who has 192.168.1.x?

Tell 192.168.1.x

Explanation

ARP is used in IPv4 local networks to map an IP address to a MAC address. An ARP request is broadcast to the local network, while the device owning the requested IP address responds with its MAC address.

Screenshot



The screenshot shows the Wireshark interface with the filter 'arp' applied. The packet list shows three packets: a request from fe:30:91:4a:9f:12 to Intel_d5:4c:a6, a response from Intel_d5:4c:a6 to fe:30:91:4a:9f:12, and a second request from fe:30:91:4a:9f:12 to Intel_d5:4c:a6. The packet details pane shows the 'arp' filter applied to the selected packet.

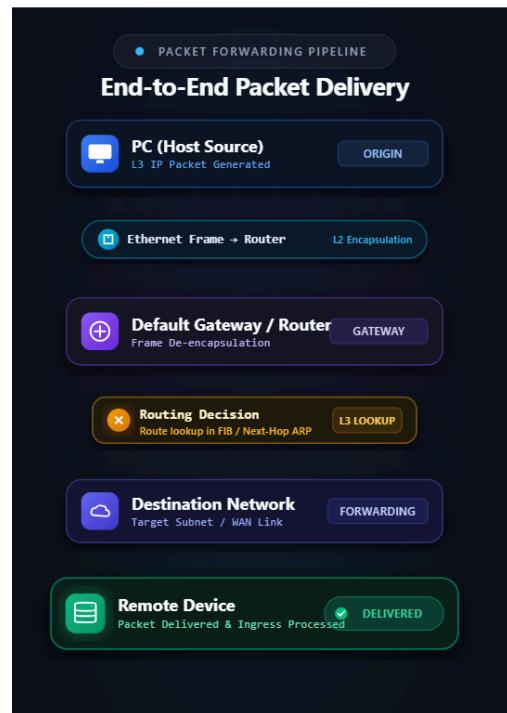
No.	Time	Source	Destination	Protocol	Length	Info
3048	12.746825100	fe:30:91:4a:9f:12	Intel_d5:4c:a6	ARP	42	Who has 10.71.124.202? Tell 10.71
3049	12.746862400	Intel_d5:4c:a6	fe:30:91:4a:9f:12	ARP	42	10.71.124.202 is at 30:89:4a:d5:4
9845	34.763268100	fe:30:91:4a:9f:12	Intel_d5:4c:a6	ARP	42	Who has 10.71.124.202? Tell 10.71

Figure 4: ARP request and response

Learning outcome

I learned how IP and MAC addresses are associated within a local network and how ARP communication can be identified in Wireshark.

The same private IP can exist on different networks without a conflict. The local network hardware delivers the frame using the MAC address. The IP address tells the network which device/network you're trying to reach, while the MAC address identifies the local network interface that should receive the frame.



If there are three separate networks - Network A, Network B, and Network C - and each network has a device using the same private IP address, 192.168.1.50, but with different MAC addresses: Network A has AA:AA:AA, Network B has BB:BB:BB, and Network C has CC:CC:CC. If your computer is connected to Network A and sends an ARP request saying “Who has 192.168.1.50?”, only the device on Network A can receive and respond to that ARP broadcast. The devices on Networks B and C never receive the request because routers do not normally forward ARP broadcasts between networks. Therefore, your computer receives only AA:AA:AA as the MAC address for 192.168.1.50. If all three devices were instead connected to the same local network, then all three could respond, creating an IP address conflict, and your computer would not be able to reliably determine which physical device you intended to reach.

10. Experiment 04 - TCP Three-Way Handshake

This should be one of the most important sections in your report.

Objective

To understand how TCP establishes a connection between a client and a server.

Use: **tcp**

TCP flags in simple terms

- **SYN** → Start a connection
- **SYN, ACK** → Accept the connection
- **ACK** → Acknowledge
- **PSH, ACK** → Send data + acknowledge
- **FIN, ACK** → "I'm finished sending; let's close the connection"
- **RST** → Immediately reset/abort the connection



Explanation

The TCP three-way handshake is used to establish a reliable connection between two endpoints. The client initially sends a SYN packet. The server responds with SYN and ACK flags. Finally, the client sends an ACK packet, completing the handshake.

Screenshot

No.	Time	Source	Destination	Protocol	Length	Info
16504	412.526134900	2407:c00:d001:489b::2600	10.71.124.202	TCP	86	63312 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
16505	412.835965000	10.71.124.202	2407:c00:d001:489b::2600	TCP	66	63313 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
16506	412.885359300	2407:c00:d001:489b::2600	10.71.124.202	TCP	86	80 → 63312 [SYN, ACK] Seq=0 Ack=1 Win=64800 Len=0 MSS=1328 SACK_PERM WS=128
16507	413.108759000	10.71.124.100	10.71.124.202	TCP	55	53 → 64573 [PSH, ACK] Seq=1 Ack=56 Win=65536 Len=1 [TCP PDU reassembled in 16513]
16508	413.108762500	23.11.38.161	10.71.124.202	TCP	66	80 → 63313 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1348 SACK_PERM WS=128
16509	413.108764300	2600:14e1:18:6d::	2407:c00:d001:489b::2600	TCP	86	[TCP Retransmission] 80 → 63312 [SYN, ACK] Seq=0 Ack=1 Win=64800 Len=0 MSS=1328 SACK_PERM WS=128
16510	413.109048200	10.71.124.202	23.11.38.161	TCP	54	63313 → 80 [ACK] Seq=1 Ack=1 Win=65280 Len=0
16511	413.109847300	10.71.124.202	23.11.38.161	HTTP	311	GET /DigiCertAssuredIDRootCA.crl HTTP/1.1
16512	413.163480900	10.71.124.202	10.71.124.100	TCP	54	64573 → 53 [ACK] Seq=56 Ack=2 Win=65280 Len=0
16513	413.166428300	10.71.124.100	10.71.124.202	DNS	332	Standard query response 0x4128 AAAA optimizationguide-pa.googleapis.com AAAA 2001:4860:4841:400:: AAAA 2001:4860:4843:400:: AAA
16514	413.170540500	10.71.124.202	10.71.124.100	TCP	54	64573 → 53 [FIN, ACK] Seq=56 Ack=280 Win=65024 Len=0
16515	413.172895100	10.71.124.100	10.71.124.202	TCP	54	53 → 64573 [FIN, ACK] Seq=280 Ack=57 Win=65536 Len=0
16516	413.172984700	10.71.124.202	10.71.124.100	TCP	54	64573 → 53 [ACK] Seq=57 Ack=281 Win=65024 Len=0
16517	413.180690900	23.11.38.161	10.71.124.202	TCP	66	[TCP Out-of-Order] 80 → 63312 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1348 SACK_PERM WS=128
16518	413.180622100	10.71.124.202	23.11.38.161	TCP	66	[TCP Dup ACK 16510#1] 63313 → 80 [ACK] Seq=259 Ack=1 Win=65280 Len=0 SIF=0 SRE=1
16519	413.520783700	2407:c00:d001:489b::2600	2407:c00:d001:489b::2600	TCP	86	51222 → 53 [SYN] Seq=0 Win=65535 Len=0 MSS=1440 WS=256 SACK_PERM
16520	413.529109800	2407:c00:d001:489b::2600	2407:c00:d001:489b::2600	TCP	86	54705 → 53 [SYN] Seq=0 Win=65535 Len=0 MSS=1440 WS=256 SACK_PERM
16521	413.529303700	2407:c00:d001:489b::2600	2407:c00:d001:489b::2600	TCP	86	53398 → 53 [SYN] Seq=0 Win=65535 Len=0 MSS=1440 WS=256 SACK_PERM
16522	413.533958000	2407:c00:d001:489b::2600	2407:c00:d001:489b::2600	TCP	86	53 → 51222 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1440 SACK_PERM WS=512
16523	413.534150000	2407:c00:d001:489b::2600	2407:c00:d001:489b::2600	TCP	74	51222 → 53 [ACK] Seq=1 Ack=1 Win=65280 Len=0
16524	413.534232000	2407:c00:d001:489b::2600	2407:c00:d001:489b::2600	TCP	86	53 → 53398 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1440 SACK_PERM WS=512
16525	413.534281000	2407:c00:d001:489b::2600	2407:c00:d001:489b::2600	TCP	74	53398 → 53 [ACK] Seq=1 Ack=1 Win=65280 Len=0

Figure 5: TCP three-way handshake captured using Wireshark

It may contain URLs, hostnames, usernames, query data, or other payload-related information in the info column.

Learning outcome

This experiment helped me understand how TCP connections are established and how TCP flags can be examined using Wireshark.

`tcp.flags.syn == 1 && tcp.flags.ack == 0` - Initiate a new connection

11. Experiment 05 - HTTPS/TLS Analysis

Objective

To understand what network information can be observed when accessing websites using HTTPS.

Visit a website such as:

<https://example.com>

Use : `tls` or `tcp.port == 443`

Observation

Look for:

TLS packets
TCP packets
Destination port 443
TLS handshake information

learned

HTTPS encrypts application-level communication between the client and server. Although the network packets themselves can be captured, the encrypted application data cannot normally be read directly as plain text from the packet capture.

Screenshot

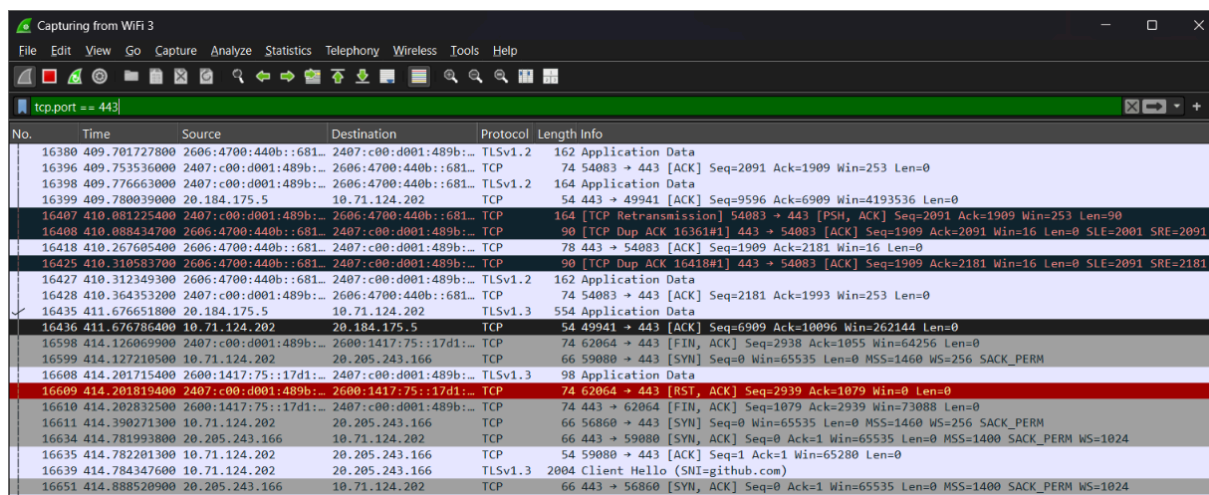


Figure 6: TLS/HTTPS traffic captured using Wireshark

12. Experiment 06 - HTTP vs HTTPS

If you can safely perform this using a test site/environment, compare HTTP and HTTPS.

Create a table:

Feature	HTTP	HTTPS
Default port	80	443
Encryption	No	Yes, using TLS

Traffic visibility	More information may be visible	Application data is encrypted
Security	Less secure	Provides encrypted communication
Wireshark analysis	HTTP fields can be visible	TLS-encrypted traffic is observed

- Port 443 = Secure Web traffic.
- Port 80 = Unsecure Web traffic.
- Other TCP Ports = Thousands of other apps (like games, email, and files), many of which use TLS to stay secure too.
- ICMP = Zero ports. It operates completely independently just to report network errors.

13. Experiment 07 - Windows Network Connections

This is a nice addition because you're using Windows directly.

Open CMD:

netstat -ano

Record some observations.

For example:

Protocol	Local Address	Foreign Address	State
TCP	Local IP:xxxxx	Remote IP:443	ESTABLISHED

Then explain:

The netstat command provides information about active network connections on the Windows system. This information can be compared with packet-level information captured in Wireshark.

This shows that you understand both:

Operating-system-level networking and packet-level networking.

Connection States

- **LISTENING** = Open door, waiting for a guest.
- **ESTABLISHED** = Guest is inside, actively talking.
- **TIME_WAIT / CLOSE_WAIT** = Guest just left, cleaning up the room.
- **SYN_SENT** = Knocking on someone else's door.

The image displays two windows. The top window is Wireshark, showing a packet capture from a WiFi interface. The filter is set to 'ip.addr == 10.71.124.202 && ip.addr == 4.145.79.82 && tcp.port == 53771 && tcp.port == 443'. The packet list shows several TCP packets, including a sequence of 'Keep-Alive' messages and an 'Application Data' packet. The bottom window is a Windows Command Prompt running the command 'C:\Users\ASUS>netstat -ano'. It shows a list of active connections with columns for Protocol, Local Address, Foreign Address, State, and PID. The connections include multiple listening ports (0.0.0.0:135, 0.0.0.0:445, 0.0.0.0:982, 0.0.0.0:912, 0.0.0.0:2179, 0.0.0.0:3306, 0.0.0.0:5848, 0.0.0.0:33060, 0.0.0.0:49664, 0.0.0.0:49665, 0.0.0.0:49666, 0.0.0.0:49667, 0.0.0.0:49668, 0.0.0.0:49676, 0.0.0.0:59168, 10.71.124.282:139, 10.71.124.282:49670, 10.71.124.282:58961, 10.71.124.282:51899, 10.71.124.282:53771) and one established connection to 4.145.79.82:443 on port 53771.

No.	Time	Source	Destination	Protocol	Length	Info
14271	233.152344700	10.71.124.202	4.145.79.82	TCP	55	53771 → 443 [ACK] Seq=1 Ack=1 Win=254 Len=1
14272	233.291971900	4.145.79.82	10.71.124.202	TCP	66	443 → 53771 [ACK] Seq=1 Ack=2 Win=8058 Len=0 SLE=1 SRE=2
19477	478.380414900	10.71.124.202	4.145.79.82	TCP	55	[TCP Keep-Alive] 53771 → 443 [ACK] Seq=1 Ack=1 Win=254 Len=0
19478	478.645551400	4.145.79.82	10.71.124.202	TCP	66	[TCP Keep-Alive ACK] 443 → 53771 [ACK] Seq=1 Ack=2 Win=8058 Len=0
21539	723.659166600	10.71.124.202	4.145.79.82	TCP	55	[TCP Keep-Alive] 53771 → 443 [ACK] Seq=1 Ack=1 Win=254 Len=0
21540	723.734972800	4.145.79.82	10.71.124.202	TCP	66	[TCP Keep-Alive ACK] 443 → 53771 [ACK] Seq=1 Ack=2 Win=8058 Len=0
21545	733.117146300	10.71.124.202	4.145.79.82	TLSv1.2	154	Application Data
21546	733.182406800	4.145.79.82	10.71.124.202	TLSv1.2	225	Application Data
21547	733.225187900	10.71.124.202	4.145.79.82	TCP	54	53771 → 443 [ACK] Seq=102 Ack=172 Win=253 Len=0
22549	978.185458600	10.71.124.202	4.145.79.82	TCP	55	[TCP Keep-Alive] 53771 → 443 [ACK] Seq=101 Ack=172 Win=253 Len=0
22550	978.362972100	4.145.79.82	10.71.124.202	TCP	66	[TCP Keep-Alive ACK] 443 → 53771 [ACK] Seq=172 Ack=102 Win=253 Len=0
24688	1223.367191700	10.71.124.202	4.145.79.82	TCP	55	[TCP Keep-Alive] 53771 → 443 [ACK] Seq=101 Ack=172 Win=253 Len=0
24719	1226.378326600	10.71.124.202	4.145.79.82	TCP	55	[TCP Keep-Alive] 53771 → 443 [ACK] Seq=101 Ack=172 Win=253 Len=0
24720	1226.439867000	4.145.79.82	10.71.124.202	TCP	66	[TCP Keep-Alive ACK] 443 → 53771 [ACK] Seq=172 Ack=102 Win=253 Len=0
31607	1471.453788400	10.71.124.202	4.145.79.82	TCP	55	[TCP Keep-Alive] 53771 → 443 [ACK] Seq=101 Ack=172 Win=253 Len=0
31625	1471.540834700	4.145.79.82	10.71.124.202	TCP	66	[TCP Keep-Alive ACK] 443 → 53771 [ACK] Seq=172 Ack=102 Win=253 Len=0

```
C:\Users\ASUS>netstat -ano

Active Connections

Proto Local Address Foreign Address State PID
TCP 0.0.0.0:135 0.0.0.0:0 LISTENING 2812
TCP 0.0.0.0:445 0.0.0.0:0 LISTENING 4
TCP 0.0.0.0:982 0.0.0.0:0 LISTENING 5136
TCP 0.0.0.0:912 0.0.0.0:0 LISTENING 5136
TCP 0.0.0.0:2179 0.0.0.0:0 LISTENING 3220
TCP 0.0.0.0:3306 0.0.0.0:0 LISTENING 6392
TCP 0.0.0.0:5848 0.0.0.0:0 LISTENING 11212
TCP 0.0.0.0:33060 0.0.0.0:0 LISTENING 6392
TCP 0.0.0.0:49664 0.0.0.0:0 LISTENING 1696
TCP 0.0.0.0:49665 0.0.0.0:0 LISTENING 1520
TCP 0.0.0.0:49666 0.0.0.0:0 LISTENING 3132
TCP 0.0.0.0:49667 0.0.0.0:0 LISTENING 3348
TCP 0.0.0.0:49668 0.0.0.0:0 LISTENING 4456
TCP 0.0.0.0:49676 0.0.0.0:0 LISTENING 1648
TCP 0.0.0.0:59168 0.0.0.0:0 LISTENING 20860
TCP 10.71.124.282:139 0.0.0.0:0 LISTENING 4
TCP 10.71.124.282:49670 10.71.124.100:53 TIME_WAIT 0
TCP 10.71.124.282:58961 52.110.15.182:443 ESTABLISHED 10588
TCP 10.71.124.282:51899 10.71.124.100:53 TIME_WAIT 0
TCP 10.71.124.282:53771 4.145.79.82:443 ESTABLISHED 5252
```

14. Experiment 08 - Suspicious Traffic Analysis

Repeated connection attempts to multiple ports can be an indicator of scanning activity.

Same Source IP - Port 22 / Port 80 / Port 443 / Port 3306 / Port 8080 / Port 8081

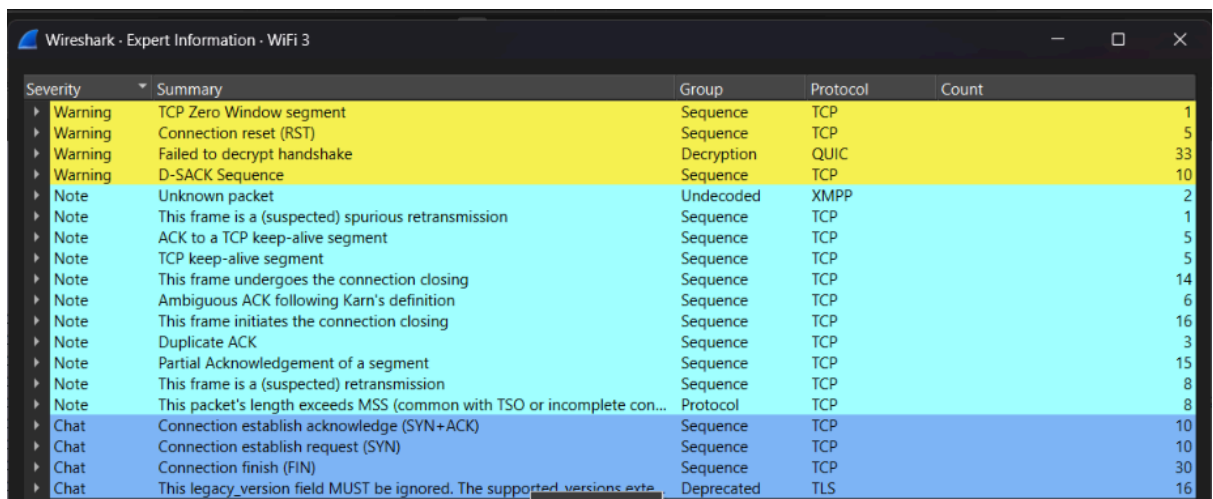
During the security analysis, repeated connection attempts to different destination ports were studied as an example of a traffic pattern that can be associated with network reconnaissance. Such patterns should be investigated further rather than automatically classified as malicious, since legitimate applications can also generate multiple connection attempts.

- Look for a massive flood of TCP RST (Reset) or TCP Retransmissions.

Port Scanning (Reconnaissance): An attacker is scanning your network to find open ports. When they hit a closed port, your firewall or host automatically sends back a TCP RST.

De-authentication / Denial of Service (DoS): Attackers can spoof IP addresses and inject TCP RST packets into an active stream to intentionally drop connections between a client and a server, disrupting service.

Security Device Intervention: If a Next-Generation Firewall (NGFW) or Intrusion Prevention System (IPS) detects malicious activity or a policy violation, it will forcefully terminate the connection by sending a TCP RST to both the client and the server.



Severity	Summary	Group	Protocol	Count
Warning	TCP Zero Window segment	Sequence	TCP	1
Warning	Connection reset (RST)	Sequence	TCP	5
Warning	Failed to decrypt handshake	Decryption	QUIC	33
Warning	D-SACK Sequence	Sequence	TCP	10
Note	Unknown packet	Undecoded	XMPP	2
Note	This frame is a (suspected) spurious retransmission	Sequence	TCP	1
Note	ACK to a TCP keep-alive segment	Sequence	TCP	5
Note	TCP keep-alive segment	Sequence	TCP	5
Note	This frame undergoes the connection closing	Sequence	TCP	14
Note	Ambiguous ACK following Karn's definition	Sequence	TCP	6
Note	This frame initiates the connection closing	Sequence	TCP	16
Note	Duplicate ACK	Sequence	TCP	3
Note	Partial Acknowledgement of a segment	Sequence	TCP	15
Note	This frame is a (suspected) retransmission	Sequence	TCP	8
Note	This packet's length exceeds MSS (common with TSO or incomplete con...	Protocol	TCP	8
Chat	Connection establish acknowledge (SYN+ACK)	Sequence	TCP	10
Chat	Connection establish request (SYN)	Sequence	TCP	10
Chat	Connection finish (FIN)	Sequence	TCP	30
Chat	This legacy_version field MUST be ignored. The supported versions exte...	Deprecated	TLS	16

15. Wireshark Filters Learned

Create a dedicated section.

Purpose	Filter
TCP	tcp
UDP	udp
DNS	dns
ARP	arp
ICMP	icmp

HTTP	http
TLS	tls
Port 80	tcp.port == 80
Port 443	tcp.port == 443
Specific IP	ip.addr == x.x.x.x
TCP SYN	tcp.flags.syn == 1

16. Key Things I Learned

Networking

I learned how devices communicate using different network protocols and how network packets contain information about the communication.

TCP

I learned how TCP establishes connections using the three-way handshake and how SYN and ACK flags can be identified.

DNS

I learned how DNS queries are generated when resolving domain names and how DNS responses provide IP address information.

ARP

I learned how ARP is used to map IPv4 addresses to MAC addresses within a local network.

HTTPS

I learned that HTTPS traffic can be captured but the application data is protected through encryption.

Security

I learned how packet-level traffic analysis can help identify unusual communication patterns and provide evidence for network security investigations.

Special Ports

Port Number	Protocol Name	Wireshark Filter	What It Does / Wireshark Behavior
53	DNS	dns	Translates domain names to IP addresses; highly readable in Wireshark.
80	HTTP	http	Unencrypted web traffic; fully viewable plaintext payloads.
443	HTTPS	tls or ssl	Encrypted web traffic; appears as scrambled data unless keys are provided.
67 / 68	DHCP	dhcp or bootp	Automatically assigns IP addresses to devices on a network.

17. Challenges Faced

During the initial experiments, it was difficult to understand the large number of packets captured by Wireshark. Display filters were therefore used to isolate specific protocols and make the analysis easier.

It was initially difficult to distinguish between TCP packets belonging to different connections. Examining source and destination IP addresses, ports, TCP flags, and packet information helped to understand individual connections.

18. Conclusion

This project provided practical experience in network traffic analysis using Wireshark. Through a series of experiments, I analyzed ICMP, DNS, ARP, TCP, HTTPS/TLS, and other network traffic. I gained a better understanding of how network communication occurs at the packet level and how protocols can be identified and investigated.

The project also introduced basic network security analysis by examining communication patterns and understanding how packet captures can provide evidence during network investigations. Overall, this project strengthened my understanding of computer networking and provided practical experience with a widely used network analysis tool.

19. Future Improvements

Since you're not using Python, you can still mention future improvements without actually implementing them.

For example:

Future improvements could include:

- Analyzing larger PCAP datasets.
- Creating a dedicated virtual network laboratory.
- Investigating additional protocols.
- Performing controlled security experiments in an isolated environment.
- Learning advanced Wireshark analysis techniques.
- Integrating packet analysis with security monitoring tools.
- Automating parts of the analysis using scripting.